

assignment 3

Link to the source code: [GitHub Repository](#)

Authors

- Wojciech Bogacz 156034
- Krzysztof Skrobala 156039

Problem Description

The problem considered in this project involves a set of nodes, each represented by three integer values: two coordinates (x,y)(x,y)(x,y) that define the node's position in a two-dimensional plane, and a cost value associated with the node. The objective is to select exactly 50% of all available nodes (if the total number of nodes is odd, the number of selected nodes is rounded up) and construct a Hamiltonian cycle — that is, a closed path visiting each selected node exactly once and returning to the starting node. The goal is to minimize the sum of two components: 1. The total length of the constructed cycle, and 2. The total cost of all selected nodes. The distances between nodes are calculated using the Euclidean metric, rounded to the nearest integer.

Implemented Algorithms

Local Search Greedy

```
def LocalSearchGreedy(instance, tour, is_selected, intra_move, intra_shuffle, random, eval_limit):
    evals = 0
    improvements = 0
    found = False

    // Prepare intra-move candidate pairs (for swapping or 2-opt)
    intra_pairs = all pairs (i, j) with 0 ≤ i < j < K
    Shuffle(intra_pairs)

    // Prepare inter-move candidates
    tour_positions = [0, K-1] shuffled
    unselected = list of nodes not in is_selected

    FOR each loop up to min(len(intra_pairs), 8) WHILE not found:
        // Randomly decide to test intra or inter move next
        if RandomNumber(0, 1) < 0.5:
            // Try intra move ...
            (i, j) = next pair
            if intra_shuffle == "nodes":
                delta = cost change from swapping nodes i, j in tour
            else:
                delta = cost change from performing 2-opt between edges (i, i-1), (j, j+1)

            evals += delta
            if delta < 0:
                swap nodes in tour
                improvements += 1
                found = True
                break
            // Try inter move ...
            if any tour position remains:
                pos = next tour position
                Shuffle(unselected)
                FOR each u in unselected:
                    delta = cost change from replacing tour[pos] with u
                    evals += delta
                    if delta < 0:
                        found = True (update tour, is_selected)
                        improvements += 1
                        found = True
                        break
            // Try later moves ...
            if any tour position remains:
                pos = next tour position
                Shuffle(unselected)
                FOR each u in unselected:
                    delta = cost change from replacing tour[pos] with u
                    evals += delta
                    if delta < 0:
                        found = True (update tour, is_selected)
                        improvements += 1
                        found = True
                        break

    if eval_limit > 0 AND evals == eval_limit:
        return (False, evals, improvements)

    return (found, evals, improvements)
```

Local Search Steepest

```
def LocalSearchSteepest(instance, tour, is_selected, intra_move, eval_limit):
    evals = 0
    improvements = 0
    best_delta = 0
    best_delta_pos = None

    // Explore intra moves ...
    FOR i FROM 0 TO K-1:
        FOR j FROM i+1 TO K-1:
            if intra_move == "nodes":
                delta = cost change from swapping positions i, j
            else:
                delta = cost change from 2-opt between edges (i, i-1) and (j, j+1)

            if delta < best_delta:
                best_delta = delta
                best_delta_pos = (i, j)

    if best_delta < 0 AND evals == eval_limit:
        return (False, evals, improvements)

    // Find the steepest move ...
    FOR each position pos in tour:
        FOR each node u not in is_selected:
            delta = cost change from replacing tour[pos] with u
            evals += delta
            if delta < best_delta:
                best_delta = delta
                best_delta_pos = (pos, u)

    if best_delta < 0 AND evals == eval_limit:
        return (False, evals, improvements)

    // Check best move ...
    if best_delta < 0:
        Apply best move to tour and is_selected
        improvements += 1
        return (True, evals, improvements)
    ELSE:
        return (False, evals, improvements)
```

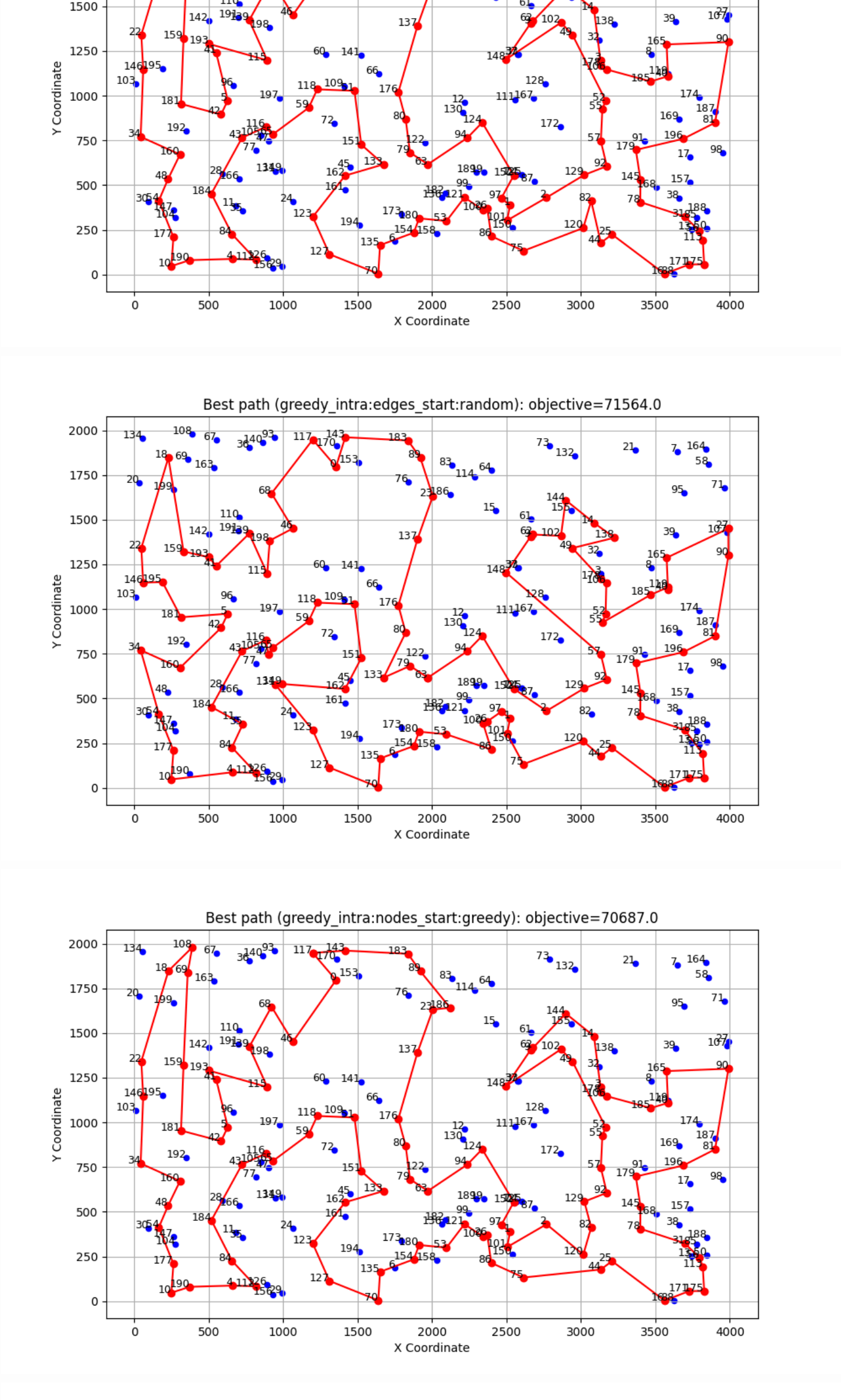
Results

Comparison of previous methods results

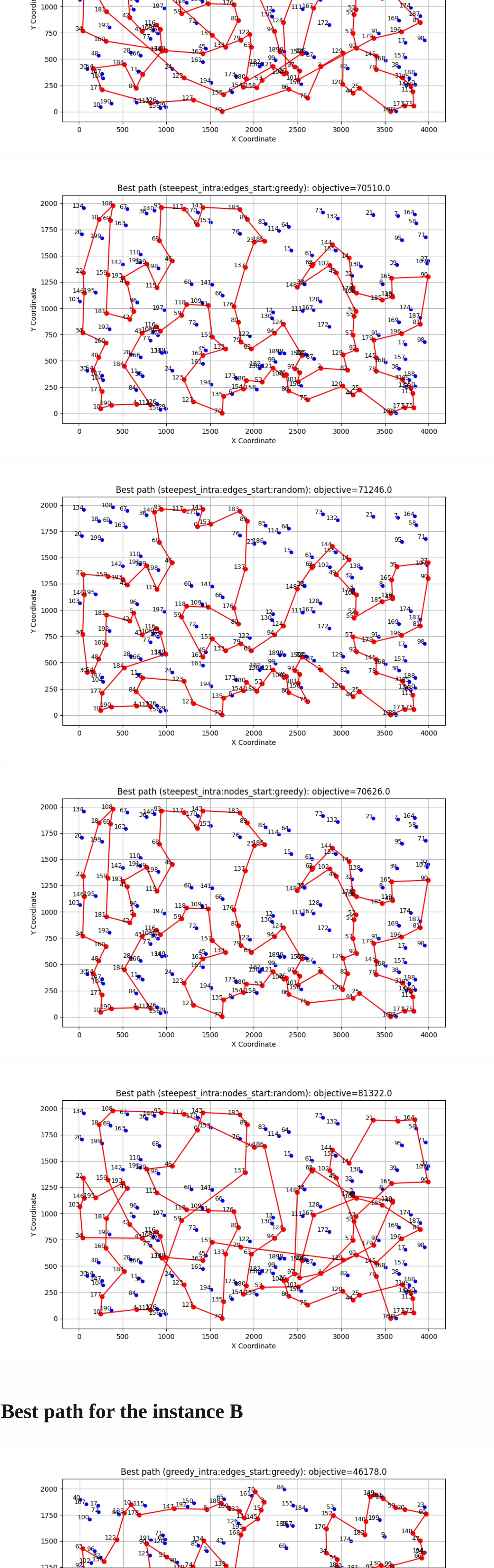
Instance A				
	Method	Best	Worst	Average
random		235308	292123	264677.17
nn_end		89198	123123	104012.785
nn_anywhere		71488	74410	72635.72
greedy cycle		72639	72639	72639.0
2-Regret Insertion		105852	123428	115474.93
Weighted Sum (α=1.00, β=1.00)		71108	73438	72130.85

Instance B				
	Method	Best	Worst	Average
random		193234	234798	214279.235
nn_end		62606	77453	69764.43
nn_anywhere		49001	57324	51400.905
greedy cycle		50243	50243	50243.0
2-Regret Insertion		66505	77072	72454.77
Weighted Sum (α=1.00, β=1.00)		47144	55700	50918.82

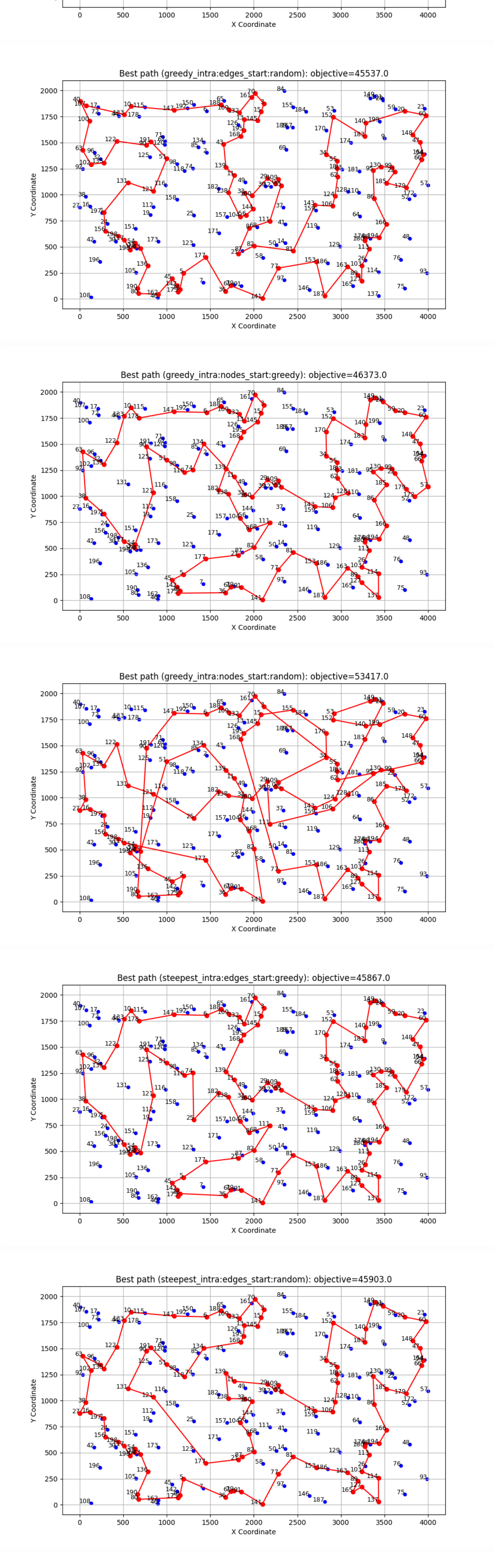
Local Search results



Best path for the instance A



Best path for the instance B



Time

Instance A				
	Method	min	max	average
greedy_intra.edges_start:greedy		0.00	0.006	0.003
greedy_intra.edges_start:random		0.062	0.099	0.080
greedy_intra.nodes_start:greedy		0.000	0.005	0.002
greedy_intra.nodes_start:random		0.066	0.118	0.085
steepest_intra.edges_start:greedy		0.001	0.005	0.002
steepest_intra.edges_start:random		0.039	0.050	0.045
steepest_intra.nodes_start:greedy		0.000	0.005	0.002
steepest_intra.nodes_start:random		0.049	0.142	0.071

Instance B				
	Method	min	max	average
greedy_intra.edges_start:greedy		0.0	0.01	0.004
greedy_intra.edges_start:random		0.064	0.102	0.081
greedy_intra.nodes_start:greedy		0.0	0.007	0.003
greedy_intra.nodes_start:random		0.058	0.107	0.082
steepest_intra.edges_start:greedy		0.001	0.01	0.004
steepest_intra.edges_start:random		0.038	0.054	0.045
steepest_intra.nodes_start:greedy		0.001	0.007	0.004
steepest_intra.nodes_start:random		0.048	0.086	0.063

Conclusions

The conducted experiments demonstrated that both the Greedy and Steepest local search algorithms effectively improved solutions for the node selection and Hamiltonian cycle optimization problem, outperforming baseline heuristics such as random, nearest neighbor, and insertion methods. Among the tested variants, approaches starting from greedy-based initial solutions consistently yielded better and more stable results compared to random starts, while the steepest variant occasionally achieved marginally better outcomes at the cost of slightly higher computation time. The methods proved to be highly efficient, with execution times in the millisecond range, and achieved significant reductions in total cost for both instances A and B. Overall, local search proved to be a robust and efficient optimization strategy, and future work could focus on integrating metaheuristic or hybrid approaches to further enhance solution quality and avoid local minima.